

00-18 code

This code only outputs 00-18 everytime.

#THIS CODE NEEDS TO BE FIXED! WE HAVE TO INCORPORATE THE LIKELIHOOD OF 19-36 IN THE OUTPUT WITHOUT AFFECTING THE OTHER PART OF THE CODE

```
import numpy as np
import pandas as pd
import json
from datetime import datetime, timedelta
import random
import ipywidgets as widgets
from IPython.display import display, clear_output
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical

# ===== Data Processing Functions =====

def parse_draw_time(time_str):
    if isinstance(time_str, str):
        try:
            return datetime.strptime(time_str, '%d-%b-%Y %I:%M:%S %p')
        except ValueError as e:
            raise ValueError(f"Time format not recognized: {time_str}, Error: {e}")
    return time_str

def process_txt_file(file_path):
    df = pd.read_csv(file_path, sep='\t', header=0,
dtype={'WinningNo.': str})
    df.columns = ['Sr. No.', 'Winning No.', 'Draw Time']
    df['Draw Time'] = df['Draw Time'].apply(parse_draw_time)
    df = df.drop_duplicates(subset=['Draw Time'],
keep='first').dropna()
    df = df.sort_values('Draw Time')
    df['Sr. No.'] = range(1, len(df) + 1)
    num_slots = len(df) // 10
    df = df.iloc[:num_slots * 10]
    slots = [df.iloc[i:i+10].to_dict(orient='records') for i in
range(0, len(df), 10)]
    return {"slots": slots}

def combine_json_data(json_list):
    all_data = []
    for json_data in json_list:
        for slot in json_data['slots']:
            all_data.extend(slot)
    df = pd.DataFrame(all_data).dropna()
```

```

df['Winning No.'] = df['Winning No.'].astype(str)
df = df.drop_duplicates(subset=['Draw Time'], keep='first')
df['Draw Time'] = df['Draw Time'].apply(parse_draw_time)
df['Draw Time'] = df['Draw Time'].apply(lambda x: x.isoformat())
df['Day'] = df['Draw Time'].apply(lambda x: x.split("T")[0])
df = df.sort_values(by=['Day', 'Draw Time'], ascending=[True,
True])
df['Sr. No.'] = range(1, len(df) + 1)
df = df.drop(columns=['Day'])
return df.to_dict(orient='records')

def process_roulette_data(txt_file_paths):
    json_list = [process_txt_file(file_path) for file_path in
txt_file_paths]
    return combine_json_data(json_list)

# ===== Load and Process Your Data =====

txt_file_paths = [
    "/content/drive/MyDrive/breaking/refined/24-03-2025.txt", # Change
it with your path
    "/content/drive/MyDrive/breaking/refined/25-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/26-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/27-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/28-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/29-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/30-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/31-03-2025.txt",
]

original_json = process_roulette_data(txt_file_paths)
#print(json.dumps(original_json[:12], indent=2, default=str)) # For
debugging

# ===== Prepare Data for LSTM =====

# Define our category conversion:
# Category 0: if the winning number is "00" or a numeric value 0-18.
# Category 1: if the winning number is between 19 and 36.
def get_category(winning_no):
    if winning_no == "00":
        return 0
    try:
        num = int(winning_no)
    except ValueError:
        return None
    return 0 if num <= 18 else 1

# Create a list of outcome categories from our processed data
data_categories = []

```

```

for record in original_json:
    cat = get_category(record["Winning No."])
    if cat is not None:
        data_categories.append(cat)

# Sequence length for our LSTM
seq_length = 5
X = []
y = []

# Build sequences from the data
for i in range(len(data_categories) - seq_length):
    X.append(data_categories[i:i+seq_length])
    y.append(data_categories[i+seq_length])

X = np.array(X)
y = np.array(y)
y = to_categorical(y, num_classes=2)
X = X.reshape(-1, seq_length, 1)

# ===== Build/Load the LSTM Model =====
# For demonstration, we build a simple LSTM model. In practice, we may
# load a pre-trained model.
model = Sequential()
model.add(LSTM(32, input_shape=(seq_length, 1)))
model.add(Dense(2, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Train the model – adjust epochs/batch_size as needed.
model.fit(X, y, epochs=10, batch_size=64, validation_split=0.2)

# ===== Compute Average Draw Interval =====
draw_times = [datetime.fromisoformat(record["Draw Time"]) for record
in original_json]
if len(draw_times) > 1:
    deltas = [(draw_times[i+1]-draw_times[i]).total_seconds() for i in
range(len(draw_times)-1)]
    avg_delta = np.mean(deltas)
else:
    avg_delta = 60 # default to 60 seconds if not enough data

# ===== Global Variables for Interactive Prediction =====
# Use the last 5 outcome categories as the initial input.
recent_categories = data_categories[-seq_length:]
# Use the last 5 actual winning numbers.
recent_numbers = [record["Winning No."] for record in original_json[-
seq_length:]]
# The last recorded draw time.
last_draw_time = datetime.fromisoformat(original_json[-1]["Draw

```

```

Time"]])

# ===== Candidate Numbers Helper =====
def get_candidate_numbers(pred_cat):
    # Category 0: "00" and numbers 1-18; Category 1: numbers 19-36.
    if pred_cat == 0:
        candidates = ["00"] + [str(i) for i in range(1, 19)]
    else:
        candidates = [str(i) for i in range(19, 37)]
    # For simplicity, choose 5 candidates at random.
    return random.sample(candidates, 5)

# ===== Prediction Function =====
def predict_next():
    # Ensure we have a sequence of length 'seq_length'
    global recent_categories, last_draw_time
    if len(recent_categories) < seq_length:
        input_seq = [recent_categories[-1]] * (seq_length -
len(recent_categories)) + recent_categories
    else:
        input_seq = recent_categories[-seq_length:]

    input_seq_np = np.array(input_seq).reshape(1, seq_length, 1)
    prediction_prob = model.predict(input_seq_np)[0]
    predicted_class = np.argmax(prediction_prob)
    candidates = get_candidate_numbers(predicted_class)
    # Predict the next draw time based on the average interval.
    predicted_draw_time = last_draw_time +
timedelta(seconds=avg_delta)
    return predicted_class, prediction_prob, candidates,
predicted_draw_time

# ===== Build the Interactive UI with ipywidgets =====

input_box = widgets.Text(
    description="Enter actual number:",
    placeholder="e.g., 17 or 00"
)
output_area = widgets.Output()

def on_submit(change):
    global recent_categories, recent_numbers, last_draw_time,
original_json
    actual_number = change.value.strip()
    input_box.value = "" # clear the input box
    # Convert the actual number to a category.
    cat = get_category(actual_number)
    if cat is None:
        with output_area:
            print("Invalid input number. Please enter a valid roulette

```

```

number (e.g., 17 or 00).")
    return
    # Append the new actual result.
    recent_categories.append(cat)
    recent_numbers.append(actual_number)
    last_draw_time = datetime.now() # use current time as the new
draw time

    # Append to the overall dataset (simulate a new record).
    new_record = {
        "Sr. No.": len(original_json) + 1,
        "Winning No.": actual_number,
        "Draw Time": last_draw_time.isoformat()
    }
    original_json.append(new_record)

    # Get the updated prediction.
    pred_cat, pred_prob, candidates, pred_draw_time = predict_next()

    with output_area:
        clear_output()
        print("Latest Actual Number Added:", actual_number)
        print("Predicted Next Category:", "00-18" if pred_cat == 0
else "19-36")
        print("Prediction Probabilities (for 00-18, 19-36):",
pred_prob)
        print("Candidate Numbers:", candidates)
        print("Predicted Draw Time:", pred_draw_time.isoformat())

input_box.on_submit(on_submit)

# Display the interactive elements.
display(input_box)
display(output_area)

# Show an initial prediction.
pred_cat, pred_prob, candidates, pred_draw_time = predict_next()
with output_area:
    print("Initial Prediction:")
    print("Predicted Next Category:", "00-18" if pred_cat == 0 else
"19-36")
    print("Prediction Probabilities (for 00-18, 19-36):", pred_prob)
    print("Candidate Numbers:", candidates)
    print("Predicted Draw Time:", pred_draw_time.isoformat())

    # In the input box input the actual number so that model gets
reinforced and appends it to the sliding window for further prediction

```

00-18 and 19-36 code

This attempt didn't succeed because the accuracy dropped as the high class (19-36) was inculcated

```
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical
from datetime import datetime
import json

# --- Data Processing Functions (as provided) ---

def parse_draw_time(time_str):
    if isinstance(time_str, str):
        try:
            return datetime.strptime(time_str, '%d-%b-%Y %I:%M:%S %p')
        except ValueError as e:
            raise ValueError(f"Time format not recognized: {time_str},
Error: {e}")
    return time_str

def process_txt_file(file_path):
    df = pd.read_csv(file_path, sep='\t', header=0, dtype={'Winning
No.': str})
    df.columns = ['Sr. No.', 'Winning No.', 'Draw Time']
    df['Draw Time'] = df['Draw Time'].apply(parse_draw_time)
    df = df.drop_duplicates(subset=['Draw Time'],
keep='first').dropna()
    df = df.sort_values('Draw Time')
    df['Sr. No.'] = range(1, len(df) + 1)
    num_slots = len(df) // 10
    df = df.iloc[:num_slots * 10]
    slots = [df.iloc[i:i+10].to_dict(orient='records') for i in
range(0, len(df), 10)]
    return {"slots": slots}

def combine_json_data(json_list):
    all_data = []
    for json_data in json_list:
        for slot in json_data['slots']:
            all_data.extend(slot)

    df = pd.DataFrame(all_data).dropna()
    df['Winning No.'] = df['Winning No.'].astype(str)
    df = df.drop_duplicates(subset=['Draw Time'], keep='first')
    df['Draw Time'] = df['Draw Time'].apply(parse_draw_time)
```

```

    df['Day'] = df['Draw Time'].apply(lambda x: x.date())
    df = df.sort_values(by=['Day', 'Draw Time'], ascending=[True,
True])
    df['Sr. No.'] = range(1, len(df) + 1)
    df = df.drop(columns=['Day'])
    return df.to_dict(orient='records')

def process_roulette_data(txt_file_paths):
    json_list = [process_txt_file(file_path) for file_path in
txt_file_paths]
    return combine_json_data(json_list)

# --- Example of loading your preprocessed data ---
txt_file_paths = [
    "/content/drive/MyDrive/breaking/refined/24-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/25-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/26-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/27-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/28-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/29-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/30-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/31-03-2025.txt",
    "/content/drive/MyDrive/breaking/refined/01-04-2025.txt",
    "/content/drive/MyDrive/breaking/refined/02-04-2025.txt",
    "/content/drive/MyDrive/breaking/refined/03-04-2025.txt",
    "/content/drive/MyDrive/breaking/refined/04-04-2025.txt",
    "/content/drive/MyDrive/breaking/refined/05-04-2025.txt",

]

# Processed data
original_json = process_roulette_data(txt_file_paths)
print(json.dumps(original_json[:5], indent=2, default=str))

import pandas as pd

# Convert JSON to DataFrame and sort by Draw Time
df = pd.DataFrame(original_json)
df['Draw Time'] = pd.to_datetime(df['Draw Time'])
df = df.sort_values('Draw Time')

# Convert winning numbers to category:
def get_category(winning_no):
    if winning_no == "00":
        return 0
    try:
        num = int(winning_no)
    except ValueError:
        return None
    return 0 if num <= 18 else 1

```

```

df['Category'] = df['Winning No.'].apply(get_category)
df = df.dropna(subset=['Category'])

# Created a sequence of categories
data = df['Category'].tolist()

# Define sequence length (last 5 outcomes)
seq_length = 5
X, y = [], []
for i in range(len(data) - seq_length):
    X.append(data[i:i+seq_length])
    y.append(data[i+seq_length])

import numpy as np
X = np.array(X).reshape(-1, seq_length, 1)
y = np.array(y)
from tensorflow.keras.utils import to_categorical
y = to_categorical(y, num_classes=2)

from datetime import timedelta

def predict_next_category(last_five):
    input_seq = np.array(last_five).reshape(1, seq_length, 1)
    prediction = model.predict(input_seq)[0]
    predicted_class = np.argmax(prediction)
    category_range = "00-18" if predicted_class == 0 else "19-36"
    return category_range, prediction

# Get the last 5 categories and last Draw Time from your DataFrame:
last_five = df['Category'].tolist()[-seq_length:]
last_draw_time = df['Draw Time'].iloc[-1]
next_draw_time = last_draw_time + timedelta(minutes=1)

predicted_range, probabilities = predict_next_category(last_five)
print("Last five categories:", last_five)
print("Predicted next category:", predicted_range)
print("Next Draw Time:", next_draw_time)
print("Prediction probabilities (for 00-18, 19-36):", probabilities)

```

This is the reinforcement cell where after the predicted output is generated, one needs to input the actual output for reinforcement

```

import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical
from datetime import timedelta

```



```

from tensorflow.keras.optimizers import Adam

# --- Load and Prepare Data ---
df = pd.DataFrame(original_json)
df['Draw Time'] = pd.to_datetime(df['Draw Time'])
df = df.sort_values('Draw Time')

def get_category(winning_no):
    if winning_no == "00":
        return 0
    try:
        num = int(winning_no)
    except ValueError:
        return None
    return 0 if num <= 18 else 1

df['Category'] = df['Winning No.'].apply(get_category)
df = df.dropna(subset=['Category'])

# --- Sequence Preparation ---
seq_length = 5
def create_sequences(df):
    X, y = [], []
    for i in range(len(df) - seq_length):
        X.append(df['Category'].iloc[i:i+seq_length].tolist())
        y.append(df['Category'].iloc[i+seq_length])
    return np.array(X).reshape(-1, seq_length, 1),
    to_categorical(np.array(y), num_classes=2)

X, y = create_sequences(df)

# --- Build LSTM Model ---
def build_model():
    model = Sequential()
    model.add(LSTM(64, input_shape=(seq_length, 1),
return_sequences=True))
    model.add(LSTM(32))
    model.add(Dense(2, activation='softmax'))
    model.compile(loss='categorical_crossentropy',
optimizer=Adam(learning_rate=0.01), metrics=['accuracy'])
    return model

model = build_model()
model.fit(X, y, epochs=10, batch_size=32, verbose=1,
validation_split=0.2)

# --- Predict the Next Draw Category ---
def predict_next_category(last_five):
    input_seq = np.array(last_five).reshape(1, seq_length, 1)
    prediction = model.predict(input_seq)[0]

```

```

predicted_class = np.argmax(prediction)
predicted_range = "00-18" if predicted_class == 0 else "19-36"
return predicted_range, prediction.tolist()

# --- Predict Top 5 Numbers (0-36) ---
def predict_top_5_numbers(last_five):
    input_seq = np.array(last_five).reshape(1, seq_length, 1)
    prediction = model.predict(input_seq)[0]
    predicted_class = np.argmax(prediction)

    if predicted_class == 0:
        candidate_pool = list(range(0, 19))
    else:
        candidate_pool = list(range(19, 37))

    top_5_numbers = np.random.choice(candidate_pool, size=5,
replace=False)
    return top_5_numbers.tolist(), predicted_class

# --- Predict Top n Categories ---
def predict_top_n_categories(last_five, n=5):
    input_seq = np.array(last_five).reshape(1, seq_length, 1)
    prediction = model.predict(input_seq)[0]
    top_n = prediction.argsort()[-n:][::-1]
    ranges = ["00-18" if idx == 0 else "19-36" for idx in top_n]
    probs = prediction[top_n]
    return list(zip(ranges, probs))

# --- Continuous Learning Loop ---
while True:
    # Get the last 5 categories and last draw time
    last_five = df['Category'].tolist()[-seq_length:]
    last_draw_time = df['Draw Time'].iloc[-1]
    next_draw_time = last_draw_time + timedelta(minutes=1)

    # Predict next category
    predicted_range, probabilities = predict_next_category(last_five)

    print("\n=====")
    print(f"Next Draw Time: {next_draw_time}")
    print(f"Predicted Category: {predicted_range}")
    print(f"Probabilities: {probabilities}")
    print("=====")

    # Predict top 5 numbers
    top_5_numbers, predicted_class = predict_top_5_numbers(last_five)
    print(f"Top 5 Predicted Numbers in range {'00-18' if
predicted_class == 0 else '19-36'}: {top_5_numbers}")

    # Show top n category predictions

```

```

predictions = predict_top_n_categories(last_five, n=5)
print(f"Predictions for {next_draw_time}:")
for i, (range_label, prob) in enumerate(predictions):
    print(f"{i+1}. {range_label} (Confidence: {prob:.4f})")

# Take user input for actual result
actual_value = input("Enter Actual Winning Number (or 'exit' to
stop): ").strip()
if actual_value.lower() == "exit":
    break

actual_category = get_category(actual_value)
if actual_category is None:
    print("Invalid input! Enter a number between 00 and 36.")
    continue

new_row = {"Draw Time": next_draw_time, "Winning No.":
actual_value, "Category": actual_category}
df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)

print("Retraining model with new data...")
X, y = create_sequences(df)
model.fit(X, y, epochs=3, batch_size=32, verbose=0)
print("Model updated! Predicting next draw...\n")

# model.save('model.keras') # RUN THIS CELL ONLY WHEN THEN OUTPUT
# AIGNS WITH EXPECTATION

```

symmetry

```

import matplotlib.pyplot as plt
import matplotlib.cm as cm
import math

# --- Spiral Visualization with Mod and Multiples ---

def visualize_spiral_with_patterns(data):
    numbers = []
    for item in data:
        try:
            num = int(item['Winning No.'])
            numbers.append(num)
        except ValueError:
            continue

    theta = [i * 0.3 for i in range(len(numbers))] # Spiral angles
    r = [i * 0.1 for i in range(len(numbers))] # Spiral radius
    x = [r[i] * math.cos(theta[i]) for i in range(len(numbers))]

```

```

y = [r[i] * math.sin(theta[i]) for i in range(len(numbers))]

# Create the plot
plt.figure(figsize=(12, 12))
plt.title("Spiral Plot Highlighting Multiples of 3, 6, 9, and 19")

# Plot all numbers as faint background
plt.scatter(x, y, c='lightgray', s=10, label='All')

# Highlight multiples
for mod_val, color, label in [(3, 'green', 'Multiples of 3'),
                              (6, 'blue', 'Multiples of 6'),
                              (9, 'purple', 'Multiples of 9'),
                              (19, 'red', 'Multiples of 19')]:
    coords = [(x[i], y[i]) for i in range(len(numbers)) if
numbers[i] % mod_val == 0]
    if coords:
        xs, ys = zip(*coords)
        plt.scatter(xs, ys, c=color, s=40, label=label)

# Mark every 19th draw (by index, not value)
every_19th = [(x[i], y[i]) for i in range(len(numbers)) if i % 19
== 0]
if every_19th:
    xs, ys = zip(*every_19th)
    plt.scatter(xs, ys, c='orange', s=50, marker='x', label='Every
19th Draw')

plt.legend()
plt.axis('equal')
plt.grid(True)
plt.show()

visualize_spiral_with_patterns(original_json)

```